

Parallel and distributed Computing



Bazam-e-Jahan

Lecturer

The University of Faisalabad

Parallel algorithms & architectures Parallel i/o

What is Parallel Computing?

- **Parallel computing** is the simultaneous execution of multiple processes or threads to solve a problem faster.
- It utilizes **multiple processors** or **cores** to divide and conquer large computational tasks.
- **Goal:** Increase efficiency by performing tasks in parallel.

- **Data Parallelism:** Divide data into smaller chunks and process them concurrently.
 - Example: Image processing, matrix operations.
- **Task Parallelism:** Split the task into independent subtasks that can be executed in parallel.
 - Example: Different steps in a simulation.
- **Pipeline Parallelism:** Divide a task into a series of stages, where each stage is performed in parallel.
 - Example: Data streaming processing.

- Parallel algorithms are designed to solve problems by dividing the work across multiple processing units.
- Categories:
 1. Divide and Conquer: Break down a problem into smaller subproblems and combine their solutions.
 - Example: Merge Sort.
 2. Greedy Algorithms: Make the locally optimal choice at each step, ensuring global optimality.
 - Example: Minimum Spanning Tree.
 3. Dynamic Programming: Solve problems by breaking them into simpler subproblems and storing results.
 - Example: Fibonacci sequence computation.
- Key Challenges: Load balancing, synchronization, and managing dependencies between tasks.

- Parallel architectures refer to the hardware configurations that support parallel computing.

1. Shared Memory Architectures

- Multiple processors share a common memory.
- Communication between processors is fast but requires synchronization.
 - Example: Multi-core CPUs.

2. Distributed Memory Architectures

- Each processor has its own private memory.
- Communication between processors is done via message passing (e.g., MPI).
 - Example: Cluster computing.

3. Hybrid Architectures

- Combination of shared and distributed memory systems.
- Often involves GPUs or accelerators working with CPUs.
- Example: HPC clusters with GPUs.

Parallelism in Practice

- Example Algorithms:

1. Matrix Multiplication:

- Divide matrix into smaller blocks and multiply them in parallel.

2. Graph Traversal:

- Use parallel BFS or DFS for processing large graphs.

3. Sorting:

- Algorithms like Parallel Merge Sort or Quick Sort for dividing and sorting in parallel.

Introduction to Parallel I/O

- Parallel I/O refers to simultaneous reading and writing of data across multiple devices or processors.
- It's crucial in handling large datasets where single-threaded I/O operations would create bottlenecks.
- Goal: Minimize I/O latency and maximize throughput by performing operations in parallel.

Why Parallel I/O?

- Large-scale data: Applications such as scientific simulations, big data analytics, and cloud computing involve handling huge amounts of data.
- Improved performance: By using multiple disks or nodes, you can perform I/O operations concurrently, speeding up data processing.
- Reduce I/O Bottlenecks: Efficient parallel I/O can help avoid the delay that single-threaded disk access can introduce.

Parallel I/O Architectures

1. Disk-Level Parallelism:

- Multiple disks are used to read and write different data segments concurrently.
- RAID (Redundant Array of Independent Disks) is a common example.

2. Network-Level Parallelism:

- Use multiple nodes in a cluster to handle different parts of the data concurrently.
- Often used in HPC systems for distributing tasks and I/O operations across nodes.

3. Hybrid I/O Systems:

- Combination of disk and network-level parallelism.
- Example: Distributed File Systems like HDFS (Hadoop Distributed File System)

1. **Data Partitioning:** Divide the data into chunks and assign different chunks to different processes or nodes.

- Example: Splitting a large file into blocks for parallel reading/writing.

2. **Buffering:** Multiple threads or processes can use local buffers to store data temporarily while I/O operations are being performed.

- Example: Double buffering in video processing.

3. **Pipelining:** Start I/O operations on one part of the data while processing other parts.

- Example: Streaming data analysis.

Challenges in Parallel I/O

- **Data Consistency:** Ensuring that concurrent reads and writes to shared resources do not cause data corruption.
- **Synchronization:** Managing when different processes or nodes should access data.
- **Scalability:** Ensuring the system can scale as the number of processes or nodes increases.

- **Scientific Simulations:** Weather prediction, molecular dynamics, simulations of physical systems.
- **Big Data Processing:** Distributed file systems like HDFS or Amazon S3 for storing and accessing large datasets.
- **Cloud Computing:** Parallel I/O techniques in managing virtual machines, distributed databases, and storage systems.

Kindly read the book before and after the lecture, if some terms/ words are not covered in the lecture, if appear in exam it might not appear strange or out of course.